

MATERIALIS

Documentação Técnica — DevOps

Aplicação, Roteiro de Testes e Artefatos Kubernetes

Repositório: github.com/MariaEduarda-Moura/Materialis

Autora: Maria Eduarda Moura

2026

1. Visão Geral da Aplicação

Materialis é uma aplicação web desenvolvida em Java com Spring Boot, seguindo o padrão arquitetural MVC, que permite a estudantes compartilhar e solicitar o empréstimo de recursos acadêmicos (livros, calculadoras, kits de eletrônica, entre outros) de forma simples e segura, promovendo a economia compartilhada dentro do ambiente universitário.

1.1. Principais Funcionalidades

- Cadastro de usuários com dois papéis (ADMIN e STUDENT), com autenticação e autorização via Spring Security.
- Gerenciamento de materiais: CRUD completo, com múltiplas fotos, estado de conservação e categorização.
- Busca e filtragem pública de materiais, por categoria, palavra-chave e localização aproximada.
- Solicitações de empréstimo, com fluxo de pedido, aprovação/recusa, sugestão de datas alternativas e notificação por e-mail.
- Agendamento de empréstimos, com controle de conflitos de calendário (um empréstimo ativo por vez).
- Avaliações pós-empréstimo, com nota por estrelas e comentários.
- Internacionalização (i18n) em Português e Inglês.

1.2. Tecnologias Utilizadas

Camada	Tecnologia
Backend	Java 17 + Spring Boot (MVC, Data JPA, Security, Mail)
Frontend	Thymeleaf + JavaScript
Banco de Dados	MySQL 8.0 (via Spring Data JPA)
Build	Maven
Containerização	Docker (multi-stage build)
Orquestração	Kubernetes (Minikube) + Helm Chart
Ingress	NGINX Ingress Controller
E-mail (ambiente de teste)	Mailpit (captura de e-mails SMTP para testes)
Controle de Versão	Git / GitHub

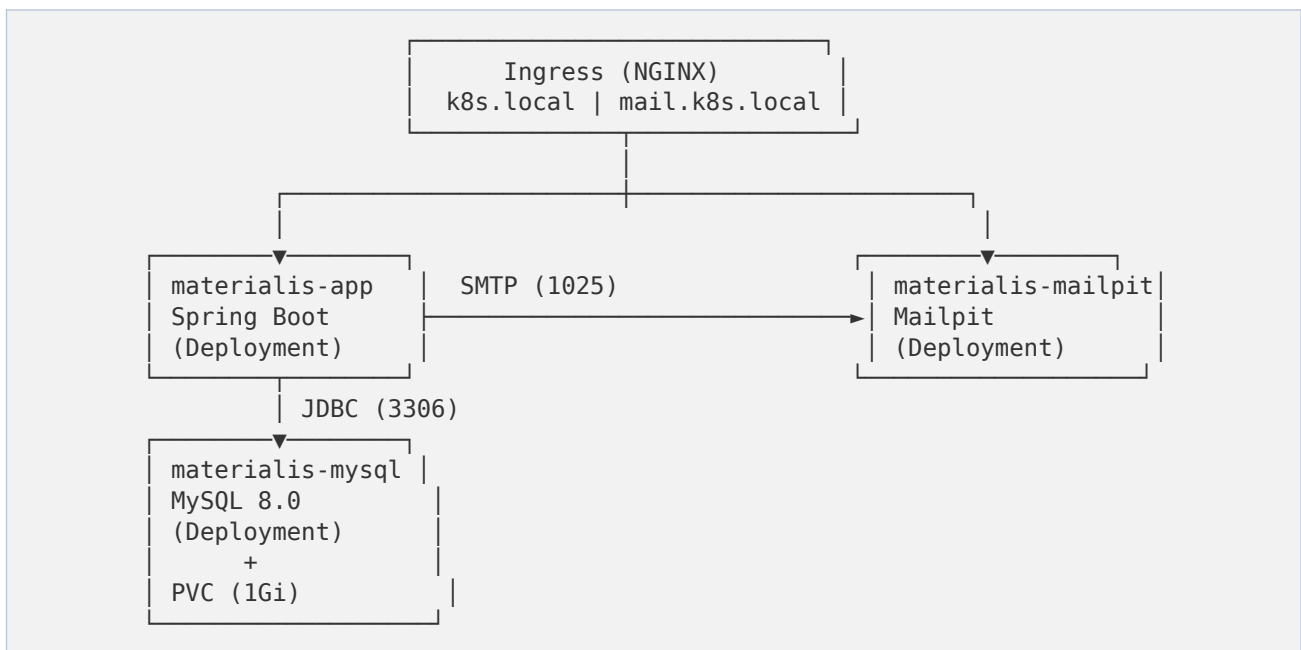
1.3. Arquitetura de Componentes e Containers

A aplicação é composta por três componentes principais, cada um isolado em seu próprio container, orquestrados via Kubernetes no cluster Minikube. A comunicação entre eles ocorre inteiramente dentro do cluster, através de Services internos (ClusterIP), e o acesso externo é feito por meio de um Ingress NGINX.

Componente	Imagem	Porta(s)	Responsabilidade
materialis-app	mariaeduardamoura/materialis:latest	8080	Aplicação Spring Boot (backend + frontend Thymeleaf). Contém toda a lógica de negócio: usuários, materiais, empréstimos e avaliações.
materialis-mysql	mysql:8.0	3306	Banco de dados relacional, com

Componente	Imagem	Porta(s)	Responsabilidade
			volume persistente para armazenar usuários, materiais, empréstimos e avaliações.
materialis-mailpit	axllent/mailpit:latest	1025 (SMTP) / 8025 (Web)	Servidor SMTP de teste que captura os e-mails enviados pela aplicação (notificações de empréstimo) e os exibe em uma interface web, sem enviá-los de fato.

1.4. Diagrama de Componentes



1.5. Fluxo de Inicialização

O Deployment do backend (materialis-app) possui um initContainer (wait-for-mysql) que aguarda ativamente o banco de dados responder ao mysqladmin ping antes de permitir que o container principal da aplicação seja iniciado. Isso evita falhas de conexão na subida do Spring Boot, garantindo que o MySQL já esteja pronto para aceitar conexões.

2. Roteiro de Testes

Este roteiro cobre a validação funcional das principais jornadas do usuário, bem como a verificação da infraestrutura implantada no Kubernetes. Os testes funcionais devem ser executados com a aplicação disponível em <http://k8s.local>, e o Mailpit em <http://mail.k8s.local>, com o comando ``minikube tunnel`` ativo.

2.1. Pré-condições Gerais

- Cluster Minikube em execução (`minikube status`) com o addon ingress habilitado.
- Release Helm instalado com sucesso (`helm upgrade --install materialis ./materialis-chart`).
- Todos os pods no namespace materialis com STATUS Running e READY 1/1 (`kubectl get pods -n materialis`).
- Arquivos hosts do WSL e do Windows atualizados com `k8s.local` e `mail.k8s.local` apontando para 127.0.0.1.
- Comando `minikube tunnel` em execução em terminal separado.

2.2. Casos de Teste Funcionais

ID	Caso de Teste / Passos	Resultado Esperado
CT-01	Cadastro de novo usuário (STUDENT) 1. Acessar http://k8s.local 2. Ir até a página de cadastro 3. Preencher nome, e-mail e senha e confirmar	Usuário é criado com papel STUDENT e consegue realizar login em seguida. (Opcional para testes gerais — a aplicação já sobe com as contas de teste da seção 4.4.)
CT-02	Login e autenticação 1. Acessar a página de login 2. Informar <code>lorena@gmail.com / 123abc</code> (ou <code>luis@gmail.com / password</code>)	Acesso autorizado via Spring Security; usuário é redirecionado à área autenticada.
CT-03	Cadastro de material 1. Logado como Lorena (<code>lorena@gmail.com / 123abc</code>), acessar 'Novo Material' 2. Preencher título, categoria, estado de conservação e fotos 3. Salvar	Material aparece na listagem pública e pode ser encontrado via busca/filtro.
CT-04	Busca e filtragem de materiais 1. Acessar listagem pública 2. Filtrar por categoria e palavra-chave	Resultados exibidos correspondem aos filtros aplicados.
CT-05	Solicitação de empréstimo 1. Logado como Luis (<code>luis@gmail.com / password</code>), selecionar o material cadastrado por Lorena 2. Solicitar empréstimo, propondo data	Solicitação registrada com status pendente; e-mail de notificação capturado no Mailpit (http://mail.k8s.local).
CT-06	Aprovação / recusa de empréstimo 1. Logado como Lorena (dona do material), acessar solicitações recebidas 2. Aprovar ou recusar, opcionalmente sugerindo data alternativa	Status da solicitação é atualizado; solicitante (Luis) recebe notificação por e-mail (visível no Mailpit).
CT-07	Conflito de agendamento 1. Tentar aprovar dois empréstimos do mesmo material em datas sobrepostas	Sistema impede a sobreposição, mantendo apenas um empréstimo ativo por vez para o material.
CT-08	Avaliação pós-empréstimo	Avaliação é associada ao

ID	Caso de Teste / Passos	Resultado Esperado
	1. Concluir o empréstimo entre Lorena e Luis 2. Registrar nota (estrelas) e comentário	material/usuário e refletida no histórico de avaliações.
CT-09	Internacionalização (i18n) 1. Alternar idioma da interface entre Português e Inglês	Textos da interface são atualizados corretamente conforme o idioma selecionado.
CT-10	Controle de acesso ADMIN vs STUDENT 1. Logar como Lorena (STUDENT) e tentar acessar rota administrativa 2. Repetir logado como admin@materialis.com / admin (ADMIN) e confirmar acesso liberado	Acesso negado (403) para papel STUDENT; acesso liberado para admin@materialis.com, conforme regras do Spring Security.

2.3. Casos de Teste de Infraestrutura (Kubernetes)

ID	Caso de Teste	Comando	Resultado Esperado
CT-I01	Todos os pods sobem e ficam prontos	kubectl get pods -n materialis	3 pods (app, mysql, mailpit) com STATUS Running e READY 1/1.
CT-I02	Backend aguarda o banco antes de iniciar	kubectl logs <pod-app> -n materialis -c wait-for-mysql	Log mostra tentativas de ping até o MySQL responder, sem erro de conexão no container principal.
CT-I03	Persistência de dados do MySQL	kubectl delete pod <pod-mysql> -n materialis (aguardar recriação)	Pod é recriado automaticamente pelo Deployment; dados anteriores permanecem (PVC materialis-mysql-data).
CT-I04	Resolução de Ingress	curl -I http://k8s.local e curl -I http://mail.k8s.local	Resposta HTTP 200 (ou redirecionamento esperado) de ambos os hosts.
CT-I05	Readiness/Liveness Probes	kubectl describe pod <pod-app> -n materialis	Nenhuma falha recorrente de probe registrada nos Events; container permanece Ready.
CT-I06	Resiliência do backend	kubectl delete pod <pod-app> -n materialis	Kubernetes recria o pod automaticamente (ReplicaSet do Deployment); aplicação volta a responder após o boot.

2.4. Comandos de Apoio para Diagnóstico

```
kubectl get pods -n materialis
kubectl describe pod <nome-do-pod> -n materialis
kubectl logs <nome-do-pod> -n materialis
kubectl logs <nome-do-pod> -n materialis --previous
kubectl get svc,ingress,pvc -n materialis
kubectl get all -n materialis
```

3. Artefatos Kubernetes

A implantação é gerenciada por um Helm Chart próprio (materialis-chart), instalado via `helm upgrade --install materialis ./materialis-chart --namespace materialis --create-namespace`. Os manifestos abaixo foram extraídos do release real em execução (`helm get manifest materialis -n materialis`), refletindo fielmente o estado implantado no cluster.

3.1. Secret

materialis-mysql-secret

Armazena de forma centralizada a senha do usuário root do MySQL, evitando que a credencial fique hardcoded nos Deployments. É referenciada tanto pelo container do MySQL quanto pelo backend (via secretKeyRef).

Campo	Valor / Descrição
kind	Secret
type	Opaque
chave	root-password
consumida por	materialis-mysql (env MYSQL_ROOT_PASSWORD) e materialis-app (env SPRING_DATASOURCE_PASSWORD)
<pre> apiVersion: v1 kind: Secret metadata: name: materialis-mysql-secret namespace: materialis type: Opaque stringData: root-password: "root" </pre>	

3.2. PersistentVolumeClaim

materialis-mysql-data

Garante a persistência dos dados do banco de dados mesmo que o pod do MySQL seja reiniciado ou recriado. O volume é montado em /var/lib/mysql dentro do container.

Campo	Valor / Descrição
kind	PersistentVolumeClaim
accessModes	ReadWriteOnce
storageClassName	standard
capacidade solicitada	1Gi
<pre> apiVersion: v1 kind: PersistentVolumeClaim metadata: name: materialis-mysql-data namespace: materialis spec: accessModes: - ReadWriteOnce </pre>	

Campo	Valor / Descrição
	<pre>storageClassName: "standard" resources: requests: storage: "1Gi"</pre>

3.3. Services (ClusterIP)

Cada componente possui um Service ClusterIP, responsável por prover um endereço DNS estável dentro do cluster (ex.: materialis-mysql, materialis-mailpit) para comunicação interna entre os pods.

Service	Portas	Seletor (app.kubernetes.io/name)	Consumido por
materialis-app	8080 (http)	materialis-app	Ingress externo (rota /)
materialis-mysql	3306 (mysql)	materialis-mysql	materialis-app (JDBC) e initContainer wait-for-mysql
materialis-mailpit	1025 (smtp) / 8025 (http)	materialis-mailpit	materialis-app (SMTP) e Ingress externo (rota /mail e mail.k8s.local)

3.4. Deployments

Os três Deployments controlam o ciclo de vida dos pods, garantindo réplica única de cada componente (réplicas: 1) e recriação automática em caso de falha.

materialis-app

Campo	Valor
Imagem	mariaeduardamoura/materialis:latest
Réplicas	1
initContainer	wait-for-mysql — aguarda o MySQL responder (mysqladmin ping) antes de liberar o container principal
Porta do container	8080 (http)
Variáveis de ambiente	SPRING_DATASOURCE_URL, SPRING_DATASOURCE_USERNAME, SPRING_DATASOURCE_PASSWORD (via Secret), SPRING_MAIL_HOST, SPRING_MAIL_PORT, timezone e locale pt_BR
readinessProbe	tcpSocket:8080 — initialDelay 30s, período 10s, failureThreshold 6
livenessProbe	tcpSocket:8080 — initialDelay 60s, período 20s, failureThreshold 6

materialis-mysql

Campo	Valor
Imagem	mysql:8.0
Réplicas	1
Porta do container	3306 (mysql)

Campo	Valor
Variáveis de ambiente	MYSQL_DATABASE=materialis, MYSQL_ROOT_PASSWORD (via Secret)
Volume	mysql-data → PVC materialis-mysql-data, montado em /var/lib/mysql
readinessProbe / livenessProbe	exec mysqladmin ping — garante que o banco só é considerado pronto quando aceita conexões

materialis-mailpit

Campo	Valor
Imagem	axllent/mailpit:latest
Réplicas	1
Portas do container	1025 (smtp) / 8025 (http)
readinessProbe / livenessProbe	tcpSocket na porta http (8025)

3.5. Ingress (NGINX)

Dois recursos Ingress expõem a aplicação e o Mailpit externamente através do NGINX Ingress Controller habilitado no Minikube (minikube addons enable ingress), usando os hosts locais k8s.local e mail.k8s.local.

materialis-app (Ingress)

Host	Path	Backend
k8s.local	/	materialis-app:8080
k8s.local	/mail	materialis-mailpit:8025

materialis-mailpit (Ingress)

Host	Path	Backend
mail.k8s.local	/	materialis-mailpit:8025

3.6. Resumo dos Recursos Implantados

Tipo	Nome	Namespace
Secret	materialis-mysql-secret	materialis
PersistentVolumeClaim	materialis-mysql-data	materialis
Service	materialis-app	materialis
Service	materialis-mysql	materialis
Service	materialis-mailpit	materialis
Deployment	materialis-app	materialis
Deployment	materialis-mysql	materialis
Deployment	materialis-mailpit	materialis
Ingress	materialis-app	materialis
Ingress	materialis-mailpit	materialis

4. Roteiro de Execução — Passo a Passo

Esta seção descreve como colocar o ambiente completo no ar, cobrindo os dois cenários mais comuns: uma máquina Linux nativa e uma máquina Windows (via WSL2). Em ambos os casos, o Minikube roda localmente — ou seja, cada pessoa que for testar precisa executar este roteiro na própria máquina.

4.1. Pré-requisitos (ambos os sistemas)

- Docker instalado e em execução.
- Minikube (v1.30+).
- kubectl.
- Helm.
- Git.

Importante

O driver "docker" do Minikube recusa ser executado como usuário root. No Linux nativo, use um usuário comum com permissão de sudo e no grupo docker. No WSL, evite o usuário root padrão — crie um usuário próprio (ver seção 4.3).

4.2. Cenário A — Linux Nativo (Ubuntu/Debian)

Passos para quem já está direto em um sistema Linux, fora do Windows/WSL.

Passo 1 — Clonar o repositório

```
git clone https://github.com/MariaEduarda-Moura/Materialis.git
cd Materialis
```

Passo 2 — Subir o Minikube, o Ingress e atualizar o hosts

No Linux nativo existe apenas um arquivo /etc/hosts (não há separação como no WSL/Windows), então a flag --update-hosts já resolve tudo em um único passo:

```
chmod +x *.sh
./setup-minikube.sh --update-hosts
```

Esse comando pode pedir a senha do usuário (sudo) para editar o /etc/hosts, adicionando:

```
127.0.0.1 k8s.local
127.0.0.1 mail.k8s.local
```

Passo 3 — Abrir o túnel do Minikube

Em um novo terminal (deixe aberto durante todo o uso):

```
minikube tunnel
```

Passo 4 — Build da imagem Docker

```
docker login # apenas quem for enviar uma nova imagem
./build.sh
```

Passo 5 — Deploy no Kubernetes

```
./deploy.sh  
kubectl get pods -n materialis # aguardar READY 1/1 em todos
```

Passo 6 — Acessar

- Aplicação: <http://k8s.local>
- Mailpit (e-mails de teste): <http://mail.k8s.local>

4.3. Cenário B — Windows (via WSL2 + Ubuntu)

No Windows, todos os comandos Docker/Minikube/kubectI/Helm rodam dentro do WSL (subsistema Linux). O navegador, porém, roda no Windows — por isso é necessário um passo extra de hosts que não existe no Linux nativo.

Passo 1 — Instalar o WSL2 com Ubuntu (se ainda não tiver)

No PowerShell como Administrador:

```
wsl --install -d Ubuntu
```

Reinicie o computador se solicitado, e complete a criação do usuário Ubuntu quando o terminal abrir.

Passo 2 — Garantir usuário não-root no WSL

Dentro do terminal WSL, confirme o usuário ativo:

```
whoami
```

Se retornar "root", crie e configure um usuário comum:

```
adduser seu_usuario  
usermod -aG sudo seu_usuario  
usermod -aG docker seu_usuario  
echo -e "[user]\ndefault=seu_usuario" | sudo tee -a /etc/wsl.conf
```

Depois, no PowerShell do Windows (fora do WSL):

```
wsl --shutdown
```

Reabra o terminal WSL — ele deve entrar automaticamente com o novo usuário.

Passo 3 — Instalar Docker, Minikube, kubectI e Helm dentro do WSL

Siga a documentação oficial de cada ferramenta para Ubuntu/Debian (Docker Engine, Minikube, kubectI, Helm).

Passo 4 — Clonar o repositório

```
git clone https://github.com/MariaEduarda-Moura/Materialis.git  
cd Materialis
```

Passo 5 — Subir o Minikube e atualizar o hosts do WSL

```
chmod +x *.sh  
./setup-minikube.sh --update-hosts
```

Isso atualiza o /etc/hosts DENTRO do WSL. Esse arquivo é diferente do hosts do Windows — por isso o próximo passo é obrigatório.

Passo 6 — Atualizar o hosts do Windows

O navegador do Windows (Chrome, Edge, etc.) não enxerga o /etc/hosts do WSL. É necessário editar também o arquivo de hosts do Windows, em C:\Windows\System32\drivers\etc\hosts.

Opção 1 — Bloco de Notas como Administrador:

- Menu Iniciar → "Notepad" → botão direito → Executar como administrador
- Abrir C:\Windows\System32\drivers\etc\hosts (trocar filtro para "Todos os arquivos")
- Adicionar as linhas abaixo e salvar:

```
127.0.0.1 k8s.local
127.0.0.1 mail.k8s.local
```

Opção 2 — PowerShell como Administrador:

```
Add-Content -Path "C:\Windows\System32\drivers\etc\hosts" `
-Value "`n127.0.0.1 k8s.local`n127.0.0.1 mail.k8s.local"
```

Por que isso funciona sem descobrir IP do Minikube

O WSL2 tem localhost forwarding automático: uma porta exposta dentro do WSL (127.0.0.1) já fica acessível como 127.0.0.1 também no Windows. Por isso basta apontar k8s.local para 127.0.0.1 nos dois hosts, sem precisar do IP interno do Minikube.

Passo 7 — Abrir o túnel do Minikube

Em um novo terminal WSL (deixe aberto durante todo o uso):

```
minikube tunnel
```

Passo 8 — Build da imagem Docker

```
docker login # apenas quem for enviar uma nova imagem
./build.sh
```

Passo 9 — Deploy no Kubernetes

```
./deploy.sh
kubectl get pods -n materialis # aguardar READY 1/1 em todos
```

Passo 10 — Acessar (pelo navegador do Windows)

- Aplicação: <http://k8s.local>
- Mailpit (e-mails de teste): <http://mail.k8s.local>

4.4. Dados de Teste Pré-Cadastrados

A aplicação possui uma rotina de inicialização (executada automaticamente no primeiro boot do backend, de forma idempotente — verifica se o e-mail já existe antes de criar) que popula o banco com um usuário administrador e dois estudantes de exemplo. Não é necessário cadastrar nenhuma conta manualmente para começar a testar: essas contas já estão prontas assim que o pod materialis-app fica Running.

Papel	Nome	E-mail (login)	Senha	RA
ADMIN	Administrador	admin@materialis.com	admin	000000
STUDENT	Lorena	lorena@gmail.com	123abc	821239
STUDENT	Luis	luis@gmail.com	password	123456

Uso sugerido nos testes

Use a conta admin@materialis.com para validar funcionalidades administrativas (CT-10) e as contas de Lorena/Luis para simular o fluxo completo de empréstimo entre dois estudantes distintos (CT-05 a CT-08): logue como um deles para cadastrar um material e como o outro para solicitar o empréstimo, aprovar/recusar e avaliar.

4.5. Resumo — Ordem de Execução

Ordem	Comando / Ação	Onde
1	git clone ...	Terminal (Linux ou WSL)
2	./setup-minikube.sh --update-hosts	Terminal (Linux ou WSL)
3	Editar hosts do Windows (apenas cenário B)	Bloco de Notas / PowerShell (Admin)
4	minikube tunnel	Novo terminal (deixar aberto)
5	./build.sh	Terminal (Linux ou WSL)
6	./deploy.sh	Terminal (Linux ou WSL)
7	Acessar http://k8s.local	Navegador

4.6. Encerrando o ambiente

```
minikube stop      # pausa o cluster, mantém dados
minikube delete    # remove o cluster por completo
```